Binary Classification of COVID-19 Disease in CT Scans via Deep Convolutional Neural Networks

Madison Davis; Dr. Bashir Morshed

Abstract

The respiratory virus Acute Respiratory Syndrome Coronavirus 2 (SARS-CoV-2) has caused wide concern and a need to be able to accurately determine its effects on individual lung capacities. CT scans were chosen as the main datatype for determining whether a single patient had COVID-19 or had normal lung capacities: the reason for this is due to an inherent lack of CT datasets in the field and the hope to provide more background for individuals wishing to use CT scans for similar classification. In this study, a wide variety of CT scans with different formats were tested to determine the best possible method for binary classification models. A total of three datasets were tested on a two-dimensional and a three-dimensional algorithm, where each dataset had its own subsets and unique parameters. A finalized version of the three-dimensional model was shown to achieve an accuracy, precision, sensitivity, and F1 Score of 86.6071%, which is similar in its conclusions when compared to other existing 3D models.

Introduction

The recent outbreak of the coronavirus disease (COVID-19), along with its declaration as a pandemic by WHO on March 11, 2020 [1], has formulated drastic changes in the way our society operates, from both an economic and sociopolitical stance. The respiratory virus is linked to the severe Acute Respiratory Syndrome Coronavirus 2 (SARS-CoV-2), a species of coronavirus that originates from the betacoronavirus 2B lineage [2] and is most similar (96% genomic nucleotide identity) to a strain labeled BatCov RaTG13, which is most commonly found in wild bat populations [3].

Due to its presence in respiratory systems, SARS-CoV-2 is easily transmissible from asymptomatic to symptomatic patients in the form of droplets and fomites [4]. Upon close or direct contact, the droplets may be picked up by the hands and enter passageways such as the eyes, nose, or mouth. Recent studies additionally suggest that the virus is able to be harbored within the urinary tracts, fecal transmissions [5], and intestinal tracts, which increases the need to be able to identify the presence of SARS-CoV-2 before it becomes contractible. Upon contraction, COVID-19 has the ability to produce respiratory complications such as fever, dry cough, and bilateral pneumonia [6]. In more serious cases, clinical studies have identified digestive contractions such as damage to the liver [7], hypercoagulability, and thrombosis [8]. Critical cases often harbor side effects such as septic shock, coagulation dysfunction, acute respiratory distress syndrome (ARDS), and organ failure [9].

Scientific communities have endeavoured to seek out certain methods to reliably and efficiently identify the presence of SARS-CoV-2 within patients. The most popular method studies polymerase chain reaction (RT-PCR) in lower and upper respiratory systems via nasal swabs [10]. However, the RT-PCR lab equipment utilized is often expensive, and analysis is typically done under the supervision of a licensed professional, which thus limits the ability to administer such tests world-wide. Moreover, human error can produce false-positives and false-negatives when studying the health care-provided nasal swabs, which leaves room for additional spread.

Newer technologies propose more accurate testing of the presence of spikes from SARS-CoV-2, although their techniques vary in complexity and application. Non-invasive methods such as chest x-rays (CXR) or computed tomography (CT) are both viable options available in local hospitals or emergency centers. For the specifics of training deep learning algorithms to be able to identify the SARS-CoV-2 virus with high accuracy, graphical data with more sensors and interior detail would be better utilized for such a task. While CXR scans provide 2-dimensional images in which softer tissue appears a grey tone, its spatial dimensions are reduced in comparison to other methodologies. CT activities computively produce a 3-dimensional model based on a series of X-rays that are taken from different angles, and thus provide more data and regions to explore along the presumed presence of COVID-19 [11]. Moreover, a recent study utilizing CT scans found that of 41 Wuhan patients infected with the infection, 40 (98%) featured singular bilateral lung opacities findings and the presence of benign masses [12]. Additional studies found patchy or lumpy ground-glass opacities throughout the lungs, which thus makes tomography scans an effective and necessary consideration in identifying COVID-19 [13].

By identifying the strengths of prior deep learning algorithms formatted to classify CT lung imaging and understanding the inherent nature of the data being utilized, this report brings forth a novel, bilateral approach in hoping to identify COVID-19 strains located within the respiratory system. The following algorithms will be structured and elaborated on with the goal of accurately modeling binary classification procedures; both a 2D and a 3D binary classifier will be tested to identify the probability of the tomography scan containing remnants of the SARS-CoV-2 virus.

The subsequent sections outline 2) past, current, and optimal mechanisms towards utilizing convolutional neural networks in classifying CT scans; 3) data preparation and model setup; 4) evaluations of the current models, highlighting metrics and optimization-time graphs; 5) and future implications that models can help with or be slightly altered.

CNNs in Classification Models

2.1 Convolutional Neural Network Structure

The structure of a CNN enables the entire network to be able to extract certain spatial features from a particular image. Images are stored analytically as a large series of matrices, where a single matrix represents a RBG (red, green, or blue) value and each value within that matrix represents a single pixel's R/B/G value on a scale from 0-255. A general 2D CNN architecture consists of a series of convolutional and pooling layers, in which the convolutional layer undergoes a two-dimensional filter transformation to create a new image, more commonly referred to as a feature map. The kernel is typically an odd-size matrix (such as a 3x3) filled with numbers, with values such as 0, 1, or -1. The following formula demonstrates mathematically how the feature map matrix G[] with m rows and n columns is constructed. The input image is referred to as f while the kernel is referred to as h [4]:

$$G[m,n] = (f * h)[m,n] = \sum_{j}\sum_{k} h[j,k]f[m-j,n-k]$$

The dimensions of this output matrix are calculated with no padding and stride length 1 to the CNN. However, a minor setback to CNNs is due to the fact that the feature maps get progressively smaller the more times a kernel is translated across the dimensions of an input image: over time, the resulting images will become so small they have the potential to disappear and thus lose valuable information. Thus, padding is often reinforced into the input image (such as 1 additional pixel) before transformation occurs, which increases the dimensions of G[]. Moreover, by increasing stride lengths to reduce any overlapping on input images, the dimensions of G[] may also decrease. Thus, the dimensions of G[] based on these additional hyperparameters can be calculated below, where p refers to padding, f refers to the input image, and s refers to stride length [4].

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - f}{s} + 1 \right\rfloor$$

This process of a single layer within an input image towards the output feature map can be expanded on via adding on neurons within a convolutional layer. In more simpler models of neural networks, this concept of forward propagation can be better understood. The figure below provides a visual interpretation connecting the matrix approach towards CNN applications to the more traditional neural network model for a single kernel-feature map transformation [16]:

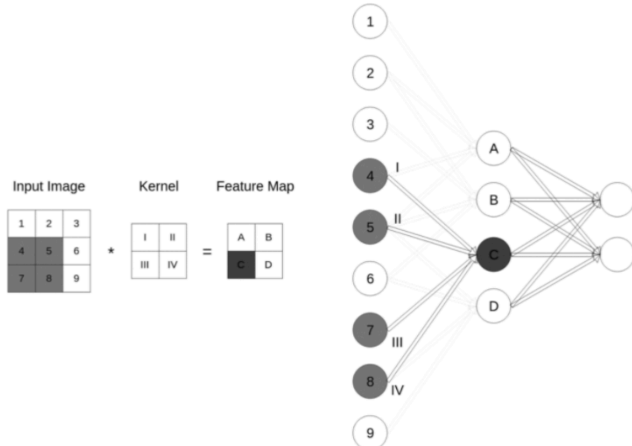


Figure 1 Places side-by-side two representations for a single kernel-mapping process in a convolutional neural network layer, the right being matrix-styled and the left being neuron-inspired

Forward propagation pertains to the forwarding of statistical information from one layer to a single neuron in the next layer. For a single neuron to calculate its own numerical value, or "activation" a_j^l, where l refers to that neuron's layer and j refers to the previous layer, it must sum together the products of each value and weight of the neuron(s) it is connected to in the previous layer, add the current layer's bias, and push this summation through a non-linear activation function f(x). The figure below shows this concept in mathematical form [9]:

$$a_i^l = f\left(\sum_{j=1}^{m} w_j^{(l)} \cdot a_{i-j+m}^{(l-1)} + b^{(l)}\right)$$

Upon receiving an output, the computer must be able to calculate the error between what is expected of that training sample and the predicted value made by the model. An Cost-Function C(x) ultimately averages the loss between all training samples and thus reflects the overall performance of the algorithm. Backpropagation, more scientifically referred to as "error back propagation" [14], is an optimization process that involves higher levels of mathematics to minimize this error.

Initializing variables and understanding their notation is crucial in successfully mastering backpropagation. For a single training example, the cost C_0 is the squared difference between the expected value y_j and the predicted value a^L_j. Referring to a single neuron in the network, the value/activation a^L_j that neuron is assigned is determined based on the non-linear function $\boldsymbol{\sigma}$ that squeezes the input z^L_j, where z^L_j refers to the forward propagation of that particular neuron.

$C_0 = (a^L_j - y_j)^2$
$z^L_j = w^L_{jk} * a^{L-1}_k + b^L_j$

$a^L_j = \boldsymbol{\sigma}(z^L_j)$

The ultimate goal of backpropagation is to mathematically calculate the gradient: a gradient is a vector, where each value in the vector refers to the cost for a single weight, bias, or activation averaged across all training data. The cost is calculated based on partial derivatives (in other words, how sensitive it is to changes in a weight/bias/activation). The cost based on a single weight for a single training example is described as follows, where the respective derivatives of the partial derivatives are described in the following [15]:

$$\partial C_0/\partial w^L_{jk} = \partial z^L_j/\partial w^L_{jk} * \partial a^L_j/\partial z^L_j * \partial C_0/\partial a^L_j$$
$$\partial C_0/\partial w^L_{jk} = 2(a^L_j - y_j) * \boldsymbol{\sigma}'(z^L_j) * a^{L-1}_k$$

If we look at the partial derivatives in order, we can make the following presumptions: $\partial z^L_j/\partial w^L_{jk} = a^{L-1}_k$ refers to how much the output neuron is influenced when we twiddle with the weight depends on how strong the previous neuron is. In other words, as once previously stated, "Neurons wired together fire together" [3]. $\partial a^L_j/\partial z^L_j = \boldsymbol{\sigma}'(z^L_j)$ refers to how much the output neuron is influenced by the number we get depends on the activation function's derivative. $\partial C_0/\partial a^L = 2(a^L_j - y_j)$ refers to how much the cost is influenced when the activation neuron is changed depends on the difference between the activation neuron and the desired output. The average cost based on that one single weight across all training examples can be calculated utilizing the function below [15]:

$$\partial C/\partial w^L = \frac{1}{n}\sum^{n-1}_{k=0} \partial C_k/\partial w^L$$

This average cost is one single value within the entire gradient vector. Every additional average cost for every single weight and bias will also be calculated, and those values will additionally be added as single entries within the gradient. The following two figures demonstrate how to calculate the costs for a single bias and activation, respectively, where the first line in each figure shows the partial derivatives and the second line in each figure shows the mathematical equivalents of those partial derivatives [15]. Ultimately, we then use the negative of the gradient vector to go in the slope of steepest descent: the opposite-signed value refers to how much to increase/decrease the weight, bias, or activation by.

$$\partial C_0/\partial b^L_j = \partial z^L_j/\partial b^L_j * \partial a^L_j/\partial z^L_j * \partial C_0/\partial a^L_j$$
$$\partial C_0/\partial b^L_j = 2(a^L_j - y_j) * \boldsymbol{\sigma}'(z^L_j)$$

$$\partial C_0/\partial a^{L-1}_k = \partial z^L_j/\partial a^{L-1}_k * \partial a^L_j/\partial z^L_j * \partial C_0/\partial a^L_j$$
$$\partial C_0/\partial a^{L-1}_k = 2(a^L_j - y_j) * \boldsymbol{\sigma}'(z^L_j) * w^L_{jk}$$

2.2 History of Convolutional Neural Networks

The history of utilizing deep learning methods for visual problems runs deep within artificial intelligence. The convolutional neural network (CNN) has the ability to focus on specific spatial areas of images and recognize minute patterns, such as edges or the developing shapes of specific lines. With deeper CNNs, these patterns are eventually grouped together in subsequent layers to recognize larger, more complex shapes, such as animal outlines or complex polygons. Early theories inspired by biological vision served to enhance the understanding of the way the sensory system worked. More specifically, studies conducted by Hubel, D.H, and Wiesel, T.N in the early 1960's focused around the structure of a cat's visual cortex [17]. Their findings suggested that the cortex consisted of two types of cells: simple cells and complex cells. The structure of simple cells allows them to take input from a field of vision and detect physical edges. Complex cells take in input from multiple simple cells that have similar oreignation but detected different edges in space [18]. These "simple-complex cell" ophthalmological studies and the general architecture of the visual cortex provided the necessary foundations for later neural networks.

It was not until 1998 when the LeNet model was introduced, whereupon the generally-known CNN structure was emulated off of. The input received by such models is typically an image with a certain number of channels, where a channel refers to the "depth" or information coded within each pixel (for instance, a pixel with RGB values would have a channel depth of 3, one for each color value). With the input, the channels are used to create a filter, which then slides over the image to extract certain values to construct a new "image", more commonly referred to as a new feature map [19]. This entire process is considered to occur within one layer, called a convolutional layer. This feature map is then put through an additional layer called a pooling layer, which simply downsizes the data to allow better efficiency. The LeNet model consists of two sets of convolutional-pooling layers, followed by an additional fully-connected layer(s) that extracts the overall pattern from the entire image [20]. The output thus consists of the best classification of the object that the model can make.

Larger and more complex versions of the LeNet model were being formatted by the 21st century. AlexNet, for example, incorporated eight subsets, five of which were convolutional-pooling layers with varying strides and three of which were fully-connected layers [21]. Moreover, the model is formatted in such a way that there would potentially be multiple convolutional layers before a pooling layer and vice versa. Convolutional neural networks in of themselves were later being formatted into more complicated algorithms such that a series of CNNs alongside max pooling layers were being considered standard single units upon which to build large scale models. Inception introduced the notion of an "Inception cell" in which a series of different CNNs with varying filter lengths (for example, 1x1, 3x3, and 5x5 convolutions) all preceded by single 1x1 filtered CNN layers would be concatenated in outputs via max pooling layers. The original Inception model also incorporated auxiliary loss structures that parted from individual Inception cells, which were designed to add on additional gradient flow (loss accounts) for deep layers in the model and thus help in optimization and backpropagation [23]. These ideas of cross-layer communication became more popular with more complicated questions, and thus many models are now beginning to incorporate denser convolutional layers via DenseNets and ResNets. Within ResNets, residual blocked layers translate information from not only one layer to the next, but “hop” over to the next two or three layers: this provides the ability to add on more convolutional layers without jeopardizing the overall performance of the network. DenseNets provide a similar approach, but focus instead on feature maps from CNN structures: feature maps are concatenated to successive layers to provide deeper layers a reference to prior information [19]. Such algorithms are highlighted due to the fact that they have been a foundational bridge for current researchers wishing to construct SARS-CoV-2 classifier models.

2.3 Prior Deep Learning Algorithms for COVID-19

Due to the rapid spread of COVID-19, the demand for advanced technical algorithms beyond traditional practices was readily increasing. In the area of artificial intelligence, multiple teams have already facilitated the use of convolutional neural networks in aiding medical teams in analyzing lung samples and traces of potential diseases.

Hassantabar et al. constructed two separate algorithms, one concerning a deep neural network and the other a convolutional neural network, to be applied to X-ray images of patient lungs in order to diagnose COVID-19 strands. With the convolutional network at an accuracy of 93.2% outperforming the deep network at an accuracy of 83.4%, little clinical studies have been enacted to provide stronger backing for a wider range of results [22]. Soares et. al [24] established an algorithm, xDNN, from a publicly available SARS-CoV-2 dataset from Sao Paulo, Brazil containing 2482 CT scans, 1252 of which had tested positive for the coronavirus. This classification system utilized a myriad of deep learning layers to be effective enough in determining whether a particular scan showed signs of the disease: like most deep learning neural network architectures, xDNN incorporated an initial feature input layer to translate the information from CT scans into neurons. By representing image (“prototype”) features as a set of density and probability distributions, the subsequent density layers identify the local peaks within these datasets. New data inputs are then compared to already labeled images in the next so-called prototype layer; the labeled prototype most similar to that of the new data is passed on to an additional layer referred to as the local decision maker layer, in which classification begins to be translated as the output. Such a model has achieved a promising F1 score of 97.31%. Zhang et. al [25] emulated a structure similar to that of traditional convolutional neural networks. In their COVSeg-NET model, which has an analytical sensitivity and specificity of 44.7% and 99.6% respectively, it is remarkable that such metrics can be analysed efficiently with a lack of data: the entire data collected consists of only 929 CT scanned images, some of which come with incomplete lung information. What the algorithm lacks in training data makes up in the architecture itself: COV-SegNET consists of 216 defined layers and more than 1.8 million adjustable parameters, and consists of patterns of 3 x 3, ELU-functional convolutional layers with max pooling. Certain convolutional layers are flattened together deeper into the algorithm to prepare for segmentation predictions. Sigmoid activation functions are incorporated near the end of the flattening layers in order to classify diseases. A considerable limitation within the study of novel fields such as the SARS-Cov-2 virus is the lack of coded or labeled information for researchers to analyze; analyzing lesions comes at a large economical cost for radiologists, especially when the profession undergoes shortages, and it is not out of the question that mis-annotations may occur. Wang et. al [26] designed a weak-learning convolutional neural network whose ultimate goal was to be able to classify diseases such as COVID-19 in a weakly-supervised manner: that is, data with little to no annotations connected to the corresponding images. In this study, 499 CT training sets and 131 CT testing samples of COVID/non-COVID patients were collected via Union Hospital, Tongji Medical College, Huazhong University of Science and Technology to produce a binary 0-1 classifier algorithm with a 0.959 ROC AUC score. Like the previous studies, deCoVNET model was designed with convolutional layers in mind. The entire algorithm can be divided into three parts: the first section, the network stem, included a 3D 5x5x7 kernel-sized convolutional layer, a batchnorm layer, and a pooling layer. The following section included a series of 3D ResBlocks, where each ResBlock contained an additional series of 3D convolutional and batchnorm layers. The final layer incorporated 3 3D convolutional layers and a softmax flattening layer for probability-classification purposes. The research included in addition to deCoVNet a UNet algorithm able to identify the specific locations of detected COVID-19 within images.

Data, Algorithm, and Methods

3.1 Overview

This report focuses on the ability to classify between COVID and Non-COVID utilizing CT scans. For this binary classification model, both a 2D and a 3D version will be tested and evaluated to understand which approach provides the best accuracy.

It is important to note that the structure of these CNN algorithms is dependent primarily on the data being imported for training purposes. CT scans are by nature 3-dimensional (separate 2D images layered on top of one another), so the classifier can naturally be set up as a three-dimensional CNN model. However, the classifier can also be trained by simply looking at each individual CT scan, which would make the model two-dimensional [38]. For the binary classification model, the dataset being used is a combination of smaller datasets, each of which contain a differing number of CT slices per patient (see the following section, 3.2 Datasets).

3.2 Overview of Datasets

The proposed models for binary classification of COVID-19 are unique in the sense both are trained from a coagulated list of previously-annotated CT scan datasets. In terms of the two-dimensional binary model, its dataset is the result of combining together three other established datasets; this was done to provide the model with a wide variety of CT scan documentations for better generalization. The first dataset imported was established by Jinyu Zhao, Yichen Zhang, Xuehai He, and Pengtao Xie of UC San Diego, and is under the name "COVID-19 Lung Scans" [27]. The dataset's images are derived from a series of previously reported papers found within medRxiv, bioRxiv, NEJM, JAMA, Lancet, and others. In total, their dataset contains 746 individual CT images from 216 patients; 397 of the images are labeled as non-COVID whereas 349 are labeled as COVID (53/47% split). The patients that were selected originate from various regions of China, including the following: Wuhan, Beijing, Shenzhen, Sichuan, Hubei, Shanghai, Zhejang, Guangdon, Hunan, Changsha, Jingmen, Qingdao, Hainan, and Xi'an. An additional number of patients were contacted via a Japanese cruise ship named "Diamond Princess". The second dataset was obtained from an eXplainable Deep Learning Approach (xDNN) written report; the dataset, named "SARS-COV-2 Ct-Scan Dataset", contains a total of 2481 CT scans, 1229 of which are labeled as non-COVID and 1252 are labeled as COVID (50/50% split). The patients consented to providing such information and their medical records originated from Sao Paulo, Brazil [28]. The final dataset under the name "Large COVID-19 CT scan slice dataset" includes data on three labeled subclasses: non-COVID, COVID, and CAP images [29]. The first two of the aforementioned subclasses were downloaded for the terms of this project. This dataset was the largest among the three due to the fact that its database itself is a coagulation of 7 additional datasets publicly available [30-36]. The data makeup of each of the 7 smaller subsets along with the entire grouped set is described in detail on the particular study's article [3], however for the purposes of this study, only the 1st dataset was utilized. It contained approximately 666 COVID-19 slices from 68 patients and 1,053 Normal Slices from 274 patients, all based in Iran.

Upon closer inspection, the "COVID-19 Lung Scans" dataset as well as the "Large COVID-19 CT scan slice dataset" contained images that appeared to contradict their initial classification: while some images were labeled as "Non-COVID", excess GGO could be easily seen and thus could distort the ability of the algorithm to correctly classify CT scans. Thus, from each individual database mentioned, each image was hand-selected to ensure the best data for training and testing. Images were omitted for various reasons, such as: too blurry; too stretched out or rotated; visualizing a slice that is too far high up into the lung chamber or too far low, which often results in large white patches in the photos; little to no GGO for a COVID-labeled photo; too much GGO for a Non-COVID-labled photo; extra text or arrows photoshopped onto the images, which impacts the pixel channel values; or too small of an image size. In the end, 74

COVID and 84 Non-COVID images were selected from the “COVID-19 Lung Scans” dataset; 98 COVID and 99 Non-COVID images were selected from the “Large COVID-19 CT scan slice dataset”, and 100 COVID and Non-COVID images were selected from the “SARS-COV-2 Ct-Scan Dataset”, which leaves a total of 555 images to split for training and testing.

The three-dimensional binary classification model utilized two of the three aforementioned datasets. The three-dimensional model requires a series of multiple images in varying depths in order to compose a single, grouped volume that represents a single CT scan of a patient. The datasets “COVID-19 Lung Scans” and “Large COVID-19 CT scan slice dataset” were annotated in a fashion such that there were more individual, CT sliced scans than patients, thus implying that a single patient’s CT scan consisted of multiple slices of CT scans, which would later be grouped into a 3D model. However, the dataset “SARS-COV-2 Ct-Scan Dataset” did not differentiate between patient number and CT scan count, and the annotations implied that each two-dimensional image would suffice in determining the presence of COVID-19. Thus, “SARS-COV-2 Ct-Scan Dataset” was not included for the training and testing of this three-dimensional binary model. Because the “COVID-19 Lung Scans” and “Large COVID-19 CT scan slice dataset” will be combined as a 3D volume of images, no images were taken out prior to concatenating. After concatenating, 68 COVID and 107 Non-COVID images were selected from the “Large COVID-19 CT scan slice dataset” while 213 COVID and 173 Non-COVID images were selected from the “COVID-19 Lung Scans”, totaling to 558 3D stacked images to be later split for training and testing.

3.2 Pre-processing Datasets

Data preparation followed a similar pattern for both models, however varying in complexity and image type. For the entire process of preparing, training, and testing the data on the model, the primary language used was Python, and additional libraries such as TensorFlow and Keras were integrated.

For the binary classification model, their respective datasets were initially mounted into a Cloud Repository for easy access and storage. Each dataset contained two subfolders, one designated for all images with a negative diagnosis whereas the other was for positive diagnosis.

For the 2D binary classification model, each image was opened and converted into float32 numpy array format. Because the datasets varied in depth, length, and pixel size, each image was standardized and resized to fit an uniform model. Standardization was calculated based on the current mean pixel values and the standard deviation to be able to fit to a 0 mean and 1 std. Because the original subsets of data contained images of varying brightness and contrast, pixel brightness had to be taken into account to normalize the data even further. Through Keras’ preprocessing ImageDataGenerator setting, the image data was reshaped using 2-98% exposure rescale. Each image was then resized to fit 200 by 200 pixels with a 3-channel depth. Later, each subset (where the images were either all COVID/NONCOVID and from a single source) was grouped into a large list containing all of the images across every dataset, and this list was utilized for testing and training. Image labels were assigned as a separate list: zeros and ones were appended based on each subset data's length and its position in the other list. By the end of the preprocessing, there were 98 COVID and 99 NONCOVID scans from the "Large COVID-19 CT scan slice dataset", 74 COVID and 84 NONCOVID scans from the "COVID-19 Lung Scans" dataset, and 100 COVID and 100 NONCOVID scans from the "SARS-COV-2 Ct-Scan Dataset". This brought the total number of scans to 555 individual slices.

For the 3D binary classification model, a similar approach was used to preprocess the data. The data was initially compounded based on its labels in a collective Google Spreadsheet, and the images were organized based on their Patient ID and relation to the entire image depth (top, middle, or bottom CT section). The data was then uploaded in Python source code, and each image was then uploaded as an array to be manipulated one by one. The data was standardized in accordance with its current mean and std values. They were then resized to a tuple of (200, 200, 3). Once every image in a single patient went through these described stages, the entire grouping of images was then reshaped to a tuple of (3, 200, 200, 3) whose values correspond to (depth, width, height, channels). By the end of the preprocessing, there were 68 COVID and 107 NONCOVID scans from the "Large COVID-19 CT scan slice dataset", and there were 213 COVID and 170 NONCOVID scans from the "COVID-19 Lung Scans" dataset. This brought the total number of scans to 558 patients.

3.3 CNN Algorithm Structures Being Tested

The 2D binary classification model is set up according to the following table. It has a set batch size of 20 and an epoch size of 9. It requires an input shape of (200, 200, 3). The output layer consists of 1 kernel, theoretically resolving to either 0 (Non-COVID) or 1 (COVID). All activation functions used relu with the exception of the output layer, which utilized the sigmoid function. The first Convolutional layer contained 32 parameters with a kernel size of 3, whereas the second and third Convolutional layers up the parameters to 64. The MaxPooling layer had a size of 2, and the first full Dense layer had 64 parameters. The "adam" optimizer was used in the last dense output layer for backpropagation. By the end of the training and testing processes, there were a total of 8,723,585 parameters, all of which were trainable.

Table 1 2D Binary Classification Algorithm Setup and Parameters

	Layer Name	Output Shape	Parameters

conv2d_42	Conv2D	(None, 198, 198, 32)	896
max_pooling2d_28	MaxPooling	(None, 99, 99, 32)	0
conv2d_43	Conv2D	(None, 97, 97, 64)	18496
max_pooling2d_29	MaxPooling	(None, 48, 48, 64)	0
conv2d_44	Conv2D	(None, 46, 46, 64)	36928
flatten_14	Flatten	(None, 135424)	0
dense_28	Dense	(None, 64)	8667200
dense_29	Dense	(None, 1)	65

The 3D binary classification model is set up according to the following table, with a batch size of 32 and epochs at 10. The Input layer required a shape of (3, 200, 200, 3) for each stacked patient image and the output layer consisted of 1 kernel, theoretically resolving to either 0 (Non-COVID) or 1 (COVID). All activation functions used relu with the exception of the output layer, which utilized the sigmoid function. In terms of the Convolutional layers, they contained 32 and 20 features, respectively. The first Conv3D contained a kernel size of (3, 3, 3) while the second contained a kernel size of (1, 2, 2), due to the rapid decrease in image size per layer. In terms of the dense layers, the first layer contained 64 features and used a L2 kernel regualizer, set at 0.1 strength. The "SGD" optimizer was used in the last dense output layer for backpropagation. By the end of the training and testing processes, there were a total of 49,680,853 parameters, all of which were trainable.

Table 2 3D Binary Classification Algorithm Setup and Parameters

	Layer Name	Output Shape	Parameters
conv3d_22	Conv3D	(None, 1, 198, 198, 32)	2624
conv3d_22	Conv3D	(None, 1, 197, 197, 20)	2580
max_pooling3d_10	MaxPooling	(None, 1, 197, 197, 20)	0
flatten_10	Flatten	(None, 776180)	0

dense_20	Dense	(None, 64)	49675584
dense_21	Dense	(None, 1)	65

Evaluation

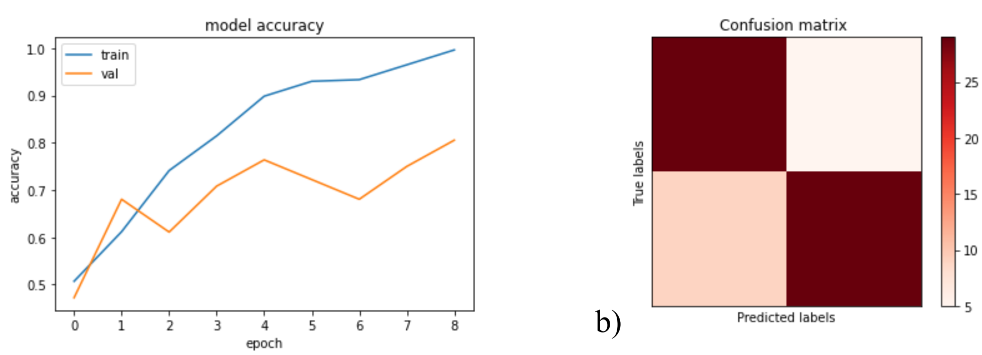
4.1 Metrics Analyzed

A multitude of metrics were analyzed to provide sufficient information for the models' current respective states. Metrics that were analyzed include the following: validation accuracy, precision, recall, F1 score, and specificity, as well as individual statistics for True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). Losses were calculated using the Sparse Categorical Cross Entropy parameter, and metrics were set to look at ["accuracy"].

Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1 score = (2 * Precision * Recall) / (Precision + Recall)
Specificity = TN / (TN + FP)

4.2 Metric Scoring and Graphs

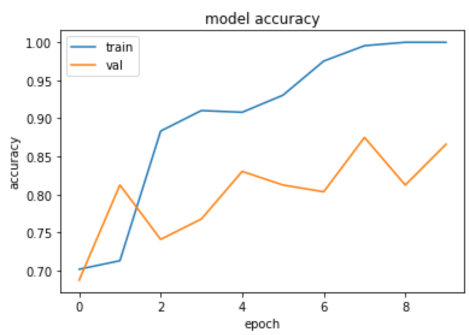
For the 2D binary model, the first table showcases the general trend between the accuracy and the epoch count. The subsequent chart highlights a confusion matrix scheme of the True Positives (top-left), False Positives (top-right), False Negatives (bottom-left), and True Negatives (bottom-right). The final model achieved an accuracy, precision, sensitivity, specificity, and F1 Score of 80.5556%. From the samples chosen for the confusion matrix, 58 were labeled as True Negative, 58 for True Positive, 14 for False Positive, and 14 for False Negative.



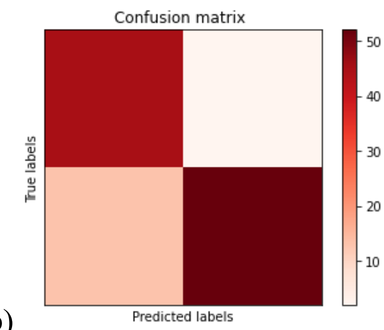
a) b)

Figure 2 2D binary model metric training showing a) accuracy and validation accuracy over eight epochs and b) a confusion matrix for a sample size of the data

For the 3D binary model, similar figures are shown below. The final model after 5 epochs achieved an accuracy, precision, sensitivity, and F1 Score of 86.6071%. From the samples chosen for the confusion matrix, 97 were labeled as True Negative, 97 for True Positive, 15 for False Positive, and 15 for False Negative.



a)



b)

Figure 3 3D binary model metric training showing a) accuracy and validation accuracy over nine epochs and b) a confusion matrix for a sample size of the data

4.3 Comparison With Other Models

For the 2D binary models, two additional models were compared on the grounds of being able to identify varying 2D CT COVID/Non-COVID scans. For the 3D binary models, four additional models were compared in a similar fashion. Their respective studies, along with their results and accuracy variabilities, are highlighted below:

Table 3 2D/3D Binary Classification Comparison to Other Algorithms on COVID-CT Data

	Model Type	Current Accuracy (%)
LUNA 16 Model [39]	3D	83
Keras CT Scans [40]	3D	83
deCovNet [41]	3D	90.1
AlexNet 3D-CNN [42]	3D	89.17
CoV SegNet Model [42]	2D	97.17
AlexNet 2D-CNN [42]	2D	87.67

Conclusion

While the 2D binary model's accuracy was less than that of its 3D counterpart, the data preprocessing for the 2-dimensional model was less strenuous and more efficient. In terms of better metrics where the expensiveness of the design is not of large importance, the 3D binary model appears to be a better and more accurate reflection of the data. With future versions of similar models, one could provide additional subsets of data and increase the patterns of Convolutional, Batch Normalization, Max Pooling, and Dropout layers to provide better accuracy and less overfitting.

References

[1] Astuti, Indwiani, and Ysrafil. "Severe Acute Respiratory Syndrome Coronavirus 2 (SARS-CoV-2): An Overview of Viral Structure and Host Response." *PubMed Central (PMC)*, NCBI, 18 Apr. 2020, www.ncbi.nlm.nih.gov/pmc/articles/PMC7165108/#bib1.

[2] Lai, Chih-Cheng, et al. "Severe Acute Respiratory Syndrome Coronavirus 2 (SARS-CoV-2) and Coronavirus Disease-2019 (COVID-19): The Epidemic and the Challenges." *PubMed*, NIH (National Library of Medicine), 17 Feb. 2020, pubmed.ncbi.nlm.nih.gov/32081636.

[3] Zhang, Chengxin, et al. "Protein Structure and Sequence Reanalysis of 2019-NCoV Genome Refutes Snakes as Its Intermediate Host and the Unique Similarity between Its Spike Protein Insertions and HIV-1." *PubMed*, NIH (National Library of Medicine), 24 Mar. 2020, pubmed.ncbi.nlm.nih.gov/32200634.

[4] Wang, Mei-Yue, et al. "SARS-CoV-2: Structure, Biology, and Structure-Based Therapeutics Development." *PubMed Central (PMC)*, NCBI, 2020, www.ncbi.nlm.nih.gov/pmc/articles/PMC7723891/#B49.

[5] Xu, Yi, et al. "Characteristics of Pediatric SARS-CoV-2 Infection and Potential Evidence for Persistent Fecal Viral Shedding." *PubMed*, NIH (National Library of Medicine), 13 Mar. 2020, pubmed.ncbi.nlm.nih.gov/32284613.

[6] Chen, Nanshan, et al. "Epidemiological and Clinical Characteristics of 99 Cases of 2019 Novel Coronavirus Pneumonia in Wuhan, China: A Descriptive Study." *PubMed*, NIH (National Library of Medicine), 15 Feb. 2020, pubmed.ncbi.nlm.nih.gov/32007143.

[7] Lee, I.-Cheng, et al. "Gastrointestinal and Liver Manifestations in Patients with COVID-19." *PubMed*, NIH (National Library of Medicine), June 2020, pubmed.ncbi.nlm.nih.gov/32243269.

[8] Haimei, M. "Pathogenesis and Treatment Strategies of COVID-19-Related Hypercoagulant and Thrombotic Complications." *PubMed*, NIH (National Library of Medicine), 2020, pubmed.ncbi.nlm.nih.gov/32722927.

[9] Zhang C., Zheng W., Huang X., Bell E. W., Zhou X., Zhang Y. (2020). Protein Structure and Sequence Reanalysis of 2019-nCoV Genome Refutes Snakes as Its Intermediate Host and the Unique Similarity between Its Spike Protein Insertions and HIV-1. *J. Proteome Res.* 19, 1351–1360. 10.1021/acs.jproteome.0c00129

[10] “EMERGENCY USE AUTHORIZATION (EUA) SUMMARY COVID-19 RT-PCR TEST (LABORATORY CORPORATION OF AMERICA).” *FDA*, 21 May 2021, www.fda.gov/media/136151/download.
[11] Los Alamos National Laboratory. “What Is the Difference between an X-Ray, a CT Scan, and an MRI?” *Bradbury Science Museum*, Triad National Security, LLC, 1 Dec. 2016, www.lanl.gov/museum/news/newsletter/2016-12/x-ray.php.
[12] Ai, T. et al. Correlation of chest ct and rt-pcr testing in coronavirus disease 2019 (covid-19) in china: a report of 1014 cases. *Radiology* 200642 (2020).
[13] Kong, W. & Agarwal, P. P. Chest imaging appearance of covid-19 infection. *Radiol. Cardiothorac. Imaging* 2, e200028 (2020).
[14] Nielsen, Michael. “Neural Networks and Deep Learning.” *Neural Networks and Deep Learning*, Determination Press, 2015, neuralnetworksanddeeplearning.com/chap2.html.
[15] Sanderson, Grant. “Backpropagation Calculus | Chapter 4, Deep Learning.” *YouTube*, uploaded by 3Blue1Brown, 3 Nov. 2017, www.youtube.com/watch?v=tIeHLnjs5U8.
[16] Skalski, Piotr. “Gentle Dive into Math Behind Convolutional Neural Networks.” *Towards Data Science*, Medium, 14 Apr. 2019, towardsdatascience.com/gentle-dive-into-math-behind-convolutional-neural-networks-79a07dd44cf9.
[17] Hubel, D. H., & Wiesel, T. N. (1962). Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. The Journal of physiology, 160(1), 106-154.
[18] Lindsay, Grace w. “Convolutional Neural Networks as a Model of the Visual System: Past, Present, and Future.” *Gatsby Computational Unit/Sainsbury Wellcome Centre*, Journal of Cognitive Neuroscience, University College London, arxiv.org/pdf/2001.07092.pdf. Accessed 5 July 2021.
[19] Jordan, Jeremy. “Common Architectures in Convolutional Neural Networks.” *Jeremy Jordan*, 20 Oct. 2018, www.jeremyjordan.me/convnet-architectures/#lenet5.
[20] Lecun, Y, Bottou, L, Bengio, Y, and Haffner, P. "Gradient-based Learning Applied to Document Recognition." Proceedings of the IEEE 86.11 (1998): 2278-324. Web.
[21] Krizhevsky, Alex, et al. “ImageNet Classification with Deep Convolutional Neural Networks.”, www.nvidia.cn/content/tesla/pdf/machine-learning/imagenet-classification-with-deep-convolutional-nn.pdf. Accessed 5 July 2021.
[22] Hassantabara, Shayan, et al. “Diagnosis and Detection of Infected Tissue of COVID-19 Patients Based on Lung x-Ray Image Using Convolutional Neural Network Approaches.” *Science Direct*, Elsevier B.V, 29 July 2020, www-sciencedirect-com.lib-e2.lib.ttu.edu/science/article/pii/S096007792030566X?via%3Dihub#!
[23] Szegedy, Christian, et al. “Rethinking the Inception Architecture for Computer Vision.” *ArXiv.Org*, Cornell University, 2 Dec. 2015, arxiv.org/abs/1512.00567.

[24] Soares, Eduardo, Angelov, Plamen, Biaso, Sarah, Higa Froes, Michele, and Kanda Abe, Daniel. "SARS-CoV-2 CT-scan dataset: A large dataset of real patients CT scans for SARS-CoV-2 identification." medRxiv (2020). doi: https://doi.org/10.1101/2020.04.24.2007858. Angelov, P., & Soares, E. (2020). Towards explainable deep neural networks (xDNN). Neural Networks, 130, 185-194.
[25] Zhang, XiaoQing, et al. "COVSeg-NET: A Deep Convolution Neural Network for COVID-19 Lung CT Image Segmentation." *Wiley Online Library*, John Wiley & Sons, Inc., 4 June 2021, onlinelibrary-wiley-com.lib-e2.lib.ttu.edu/doi/10.1002/ima.22611.
[26] Wang, Xinggang, et al. "A Weakly-Supervised Framework for COVID-19 Classification and Lesion Localization From Chest CT." *IEEE Journals & Magazine | IEEE Xplore*, IEEE, 1 Aug. 2020, ieeexplore.ieee.org/document/9097297/authors#authors.
[27] Blanche, Luis. "COVID-19 Lung CT Scans." *Kaggle*, 9 Apr. 2020, www.kaggle.com/luisblanche/covidct.
[28] Soares, Eduardo, Angelov, Plamen, Biaso, Sarah, Higa Froes, Michele, and Kanda Abe, Daniel. "SARS-CoV-2 CT-scan dataset: A large dataset of real patients CT scans for SARS-CoV-2 identification." medRxiv (2020). doi:
https://doi.org/10.1101/2020.04.24.20078584.
Angelov, P., & Soares, E. (2020). Towards explainable deep neural networks (xDNN). Neural Networks, 130, 185-194.
[29] Maftouni, M., Law, A.C, Shen, B., Zhou, Y., Yazdi, N., and Kong, Z.J. "A Robust Ensemble-Deep Learning Model for COVID-19 Diagnosis based on an Integrated CT Scan Images Database," Proceedings of the 2021 Industrial and Systems Engineering Conference, Virtual Conference, May 22-25, 2021.
[30] J. Zhao, Y. Zhang, X. He, and P. Xie, "COVID-CT-Dataset: a CT scan dataset about COVID-19," arXiv preprint arXiv:2003.13865, 2020.

[31] P. Afshar et al., "COVID-CT-MD: COVID-19 Computed Tomography (CT) Scan Dataset Applicable in Machine Learning and Deep Learning," arXiv preprint arXiv:2009.14623, 2020.

[32] J. P. Cohen, P. Morrison, L. Dao, K. Roth, T. Q. Duong, and M. Ghassemi, "Covid-19 image data collection: Prospective predictions are the future," arXiv preprint arXiv:2006.11988, 2020.

[33] S. Morozov et al., "MosMedData: Chest CT Scans With COVID-19 Related Findings Dataset," arXiv preprint arXiv:2005.06465, 2020.

[34] M. Rahimzadeh, A. Attar, and S. M. Sakhaei, "A Fully Automated Deep Learning-based Network For Detecting COVID-19 from a New And Large Lung CT Scan Dataset," medRxiv, 2020.

[35] M. Jun et al., "COVID-19 CT Lung and Infection Segmentation Dataset," Zenodo, Apr, vol. 20, 2020.

[36] "COVID-19." 2020. [Online] http://medicalsegmentation.com/covid19/ [Accessed 23 December, 2020].

[37] Ozsoz, Mehmet, et al. “Viral and Bacterial Pneumonia Detection Using Artificial Intelligence in the Era of COVID-19.” *Research Square*, 2 Sept. 2020, www.researchsquare.com/article/rs-70158/v1.

[38] Draelos, Rachel Lea Ballantyne. “Chest CT Scan Machine Learning in 5 Minutes - Towards Data Science.” *Medium*, 4 Aug. 2020, towardsdatascience.com/chest-ct-scan-machine-learning-in-5-minutes-ae7613192fdc.

[39] Joshua, Neal Eali Stephen, et al. “3D CNN with Visual Insights for Early Detection of Lung Cancer Using Gradient-Weighted Class Activation.” *Hindawi*, 12 Mar. 2021, www.hindawi.com/journals/jhe/2021/6695518.

[40] Zunair, Hasib. “Keras Documentation: 3D Image Classification from CT Scans.” *Keras*, 23 Sept. 2020, keras.io/examples/vision/3D_image_classification.

[41] ---. “A Weakly-Supervised Framework for COVID-19 Classification and Lesion Localization From Chest CT.” *IEEE Journals & Magazine | IEEE Xplore*, 1 Aug. 2020, ieeexplore.ieee.org/document/9097297.

[42] Zhang, XiaoQing, GuangYu Wang, et al. “COVSeg-NET: A Deep Convolution Neural Network for COVID-19 Lung CT Image Segmentation.” *Wiley Online Library*, John Wiley & Sons, 4 June 2021, onlinelibrary-wiley-com.lib-e2.lib.ttu.edu/doi/full/10.1002/ima.22611.